wagg Webhook Aggregator

Signed webhook ingest and capacity-bounded fan-out, backed by MariaDB

PERFORMANCE DATASHEET
release build · 2026-10-02 00:01

1 FEATURES

- **981 req/s** ingest from one client; **10544 req/s** at 64 (median of 3)
- USL ceiling **≈ 12639 req/s**; contention σ = 0.057
- p99 **< 50 ms up to 4800 req/s** at a constant arrival rate, every event delivered
- End-to-end p99 **28.7 ms** at 1000 events/s (1 KiB, OpenMessaging-style workload)
- **0 stranded and 0 lost** events across 113029 burst events, restarts and database-connection loss
- Dispatch follows Little's law: **7470 deliveries/s** at capacity 256, 20 ms service
- Claim cost flat up to 1M rows of history
- At-least-once delivery, 5 attempts, exponential backoff; terminal `SUCCESS` / `FAILED`

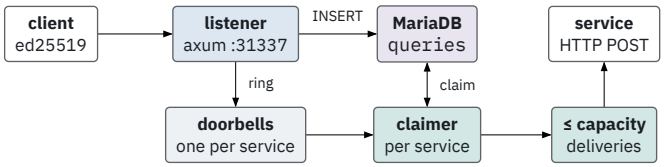


Figure 1. Functional Block Diagram

2 DESCRIPTION

wagg accepts Ed25519-signed webhooks over HTTP, stores each one durably in MariaDB before acknowledging it, and forwards it to the service mapped to the endpoint. One claimer per service drains its rows; no more than `services.capacity` deliveries per service are in flight.

The results are summarised in the formats of the tools the industry uses for these measurements: k6 end-of-test summaries and thresholds, wrk2 / HdrHistogram percentile distributions, Gatling response-time ranges, OpenMessaging Benchmark publish and end-to-end tables, the Universal Scalability Law, and Hookdeck-style delivery metrics. Test conditions are in Section 15.

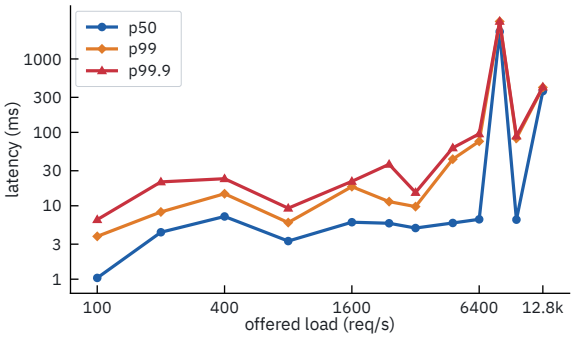


Figure 2. Ingest Latency vs Offered Load
constant arrival rate · latency from intended send time · median of 3 runs

3 SPECIFICATIONS

3.1 Fixed Parameters

Parameter	Value	Notes
Delivery attempts per event	5	then `FAILED` (dead letter)
Attempt timeout	10 s	counted as a retryable failure
Retry backoff	0.5 s × 2^n, ≤ 30 s	0.5, 1, 2, 4 s between the 5 attempts
Retryable responses	5xx, 429, network	other 4xx give up immediately
In-flight deliveries per service	≤ `services.capacity`	slot held from claim to final status write
Claim batch	≤ free slots	one `UPDATE` commit per batch
Idle re-check	5 s	claimer re-reads the table without a doorbell
Database retry backoff	0.5 s × 2^n, ≤ 5 s	claims, status writes, start-up scan
Maximum request body	2 MiB	axum default; larger bodies get HTTP 413

3.2 Performance Characteristics

Table 1. Performance characteristics. TYP = median of 3 runs; MIN / MAX over runs where shown.

Parameter	Test conditions	MIN	TYP	MAX	Unit
Ingest					
Sustained ingest	closed loop, 1 client	937	**981**	998	req/s
Sustained ingest	closed loop, 16 clients	6000	**6080**	6348	req/s
Sustained ingest	closed loop, 64 clients	10516	**10544**	10573	req/s
Throughput ceiling	USL fit over all closed-loop runs	—	**12639**	—	req/s
Ingest latency, p50	1 client	—	**0.95**	—	ms
Ingest latency, p99	1 client	—	**3.19**	—	ms
Ingest latency, p99	64 clients	—	**12.3**	—	ms
Constant rate with p99 < 50 ms	all accepted and delivered, < 0.1 % errors	4800	**—**	—	req/s
Delivery					
End-to-end latency, p50	1000 events/s, 1 KiB, 0 ms service	—	**10.1**	—	ms
End-to-end latency, p99	1000 events/s, 1 KiB, 0 ms service	—	**28.7**	—	ms
End-to-end latency, p99	2000 events/s, 1 KiB, 0 ms service	—	**37.9**	—	ms
Delivered share of accepted	any closed-loop run, after settling	100.0	**—**	—	%
Stranded events	bursts at 1–32 clients, plus restart	—	**—**	0	events
Lost or stuck events	200/s, all DB connections killed 16×	—	**—**	0	events
Dead-letter share	service fails 50 % of attempts	—	**3.0**	—	%
Dispatch					
Dispatch rate	capacity 64, 20 ms service; MAX = Little's law	—	**2501**	3200	/s
Dispatch rate	capacity 256, 20 ms service; MAX = Little's law	—	**7470**	12800	/s
Dispatch efficiency	35 points: capacity 1–64 × 2–100 ms	23.3	**87.2**	96.7	%
Cost and scaling					
Data-file fsyncs per event	1 client	—	**2.02**	—	
wagg CPU per event	1 client	—	**334**	—	µs
MariaDB CPU per event	1 client	—	**699**	—	µs
Claim lookup (`SELECT`), p50	empty table	—	**0.07**	—	ms
Claim lookup (`SELECT`), p50	1M finished rows	—	**0.05**	—	ms
Memory per queued event	16 KiB payload, service hangs	—	**212**	—	B
Heap allocations per event	1000 events/s, heaptrack	—	**408**	—	calls
Peak heap	1000 events/s, heaptrack	—	**12.5**	—	MiB
New MariaDB connections per event	16 clients, `Connections` counter	—	**2.11**	—	
SQL statements per event	16 clients, `Questions` counter	—	**12.4**	—	
Graceful shutdown	256 queued, capacity 16, 0.5 s service	—	**8.1**	—	s

3.3 Thresholds

Format: k6 thresholds — each criterion evaluated against the run named in Scope

Table 2. Pass / fail criteria

Metric	Scope	Criterion	Measured	Result
`http_req_duration`	1600/s	p(95) < 10 ms	11.6 ms	✗ **fail**
`http_req_duration`	1600/s	p(99) < 50 ms	18.2 ms	✓ **pass**
`http_req_failed`	1600/s	rate < 0.1 %	0.00 %	✓ **pass**
`http_req_duration`	4800/s	p(95) < 10 ms	18.0 ms	✗ **fail**
`http_req_duration`	4800/s	p(99) < 50 ms	43.1 ms	✓ **pass**
`http_req_failed`	4800/s	rate < 0.1 %	0.00 %	✓ **pass**
`http_req_duration`	9600/s	p(95) < 10 ms	62.7 ms	✗ **fail**
`http_req_duration`	9600/s	p(99) < 50 ms	82.8 ms	✗ **fail**
`http_req_failed`	9600/s	rate < 0.1 %	3.54 %	✗ **fail**
`e2e_latency`	1000/s, 1 KiB	p(99) < 100 ms	28.7 ms	✓ **pass**
`delivered`	1000/s, 1 KiB	= accepted	90000 events	✓ **pass**
`e2e_latency`	2000/s, 1 KiB	p(99) < 100 ms	37.9 ms	✓ **pass**
`delivered`	2000/s, 1 KiB	= accepted	180000 events	✓ **pass**
`stranded_events`	bursts, 1–32 clients	== 0	0 events	✓ **pass**
`stranded_events`	after restart	== 0	0 events	✓ **pass**
`lost_events`	DB connections killed 16×	== 0	0 events	✓ **pass**

4 LOAD TEST SUMMARY

Format: k6 end-of-test summary, `http_req_duration` columns (avg, min, med, max, p(90), p(95)) plus p(99)

Table 3. HTTP ingest request duration (ms) by scenario

Scenario	req/s	avg	min	med	max	p(90)	p(95)	p(99)	failed
closed loop, 1 VU	981	1.02	0.83	**0.95**	6.76	1.03	**1.09**	3.19	0.00 %
closed loop, 16 VUs	6080	2.63	0.98	**2.44**	24.3	3.9	**4.45**	5.98	0.00 %
closed loop, 64 VUs	10544	6.07	1.29	**5.75**	20.9	8.97	**10.0**	12.3	0.00 %
constant arrival, 200/s	200	4.58	2.88	**4.36**	23.3	5.8	**6.04**	8.24	0.00 %
constant arrival, 1600/s	1600	6.88	2.98	**5.97**	28.6	9.88	**11.6**	18.2	0.00 %
constant arrival, 4800/s	4800	11.0	3.3	**5.83**	77.4	11.5	**18.0**	43.1	0.00 %
constant arrival, 9600/s	9600	12.8	0.93	**6.47**	98.2	33.2	**62.7**	82.8	3.54 %
constant arrival, 12800/s	12800	329	164	**364**	421	392	**399**	410	0.00 %

5 THROUGHPUT SCALABILITY

Format: Universal Scalability Law (N. Gunther): $X(N) = \lambda N/(1 + \sigma(N-1) + \kappa N(N-1))$, least-squares fit over every run

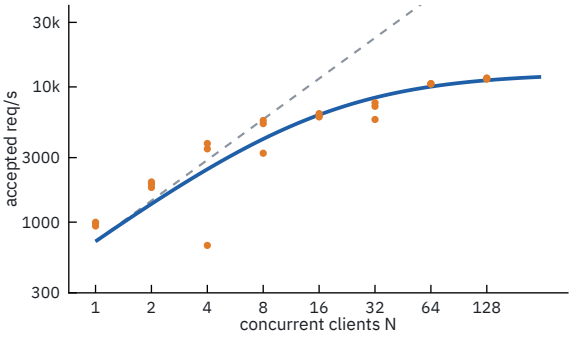


Figure 3. Throughput vs Concurrent Clients, USL Fit
closed loop · dots: individual runs · dashed: linear λN · dotted: Amdahl (κ = 0)

Table 4. USL parameters

Parameter	Value	Meaning
λ	720	req/s per client with no contention
σ	0.057	contention (serialised fraction)
κ	0	coherency (cross-talk) cost
R^2	0.935	goodness of fit
N_{max}	–	$\sqrt{(1-\sigma)/\kappa}$, peak concurrency
X_{max}	12639	λ / σ (no retrograde region)

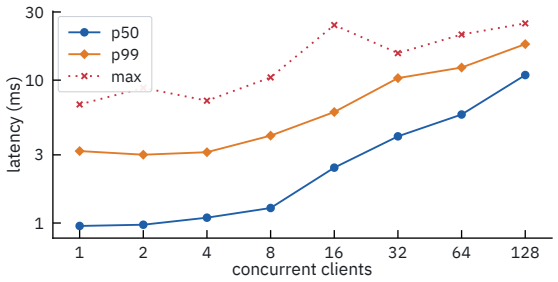


Figure 4. Ingest Latency vs Concurrent Clients
closed loop · median of 3 runs

6 LATENCY UNDER LOAD

Format: wrk2 / HdrHistogram percentile distribution (coordinated-omission corrected) and Gatling response-time charts

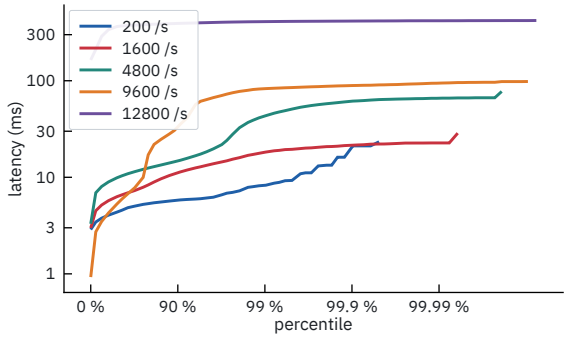


Figure 5. Latency by Percentile
HdrHistogram percentile distribution · x axis 1 / (1 – p) · one curve per arrival rate

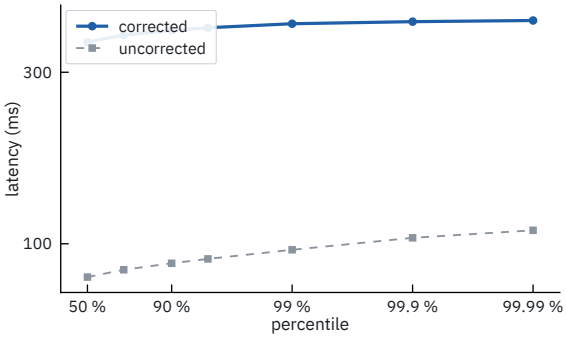


Figure 6. Coordinated Omission at 12800 req/s
corrected: from intended send time · uncorrected: from actual send (wrk "CO" view)

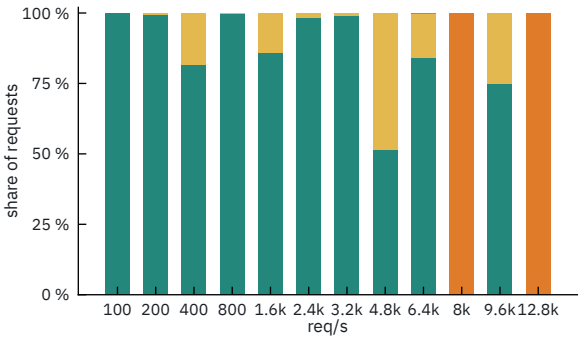


Figure 7. Response Time Ranges by Arrival Rate

Gatling ranges · t < 10 ms · 10–100 ms · ≥ 100 ms · failed

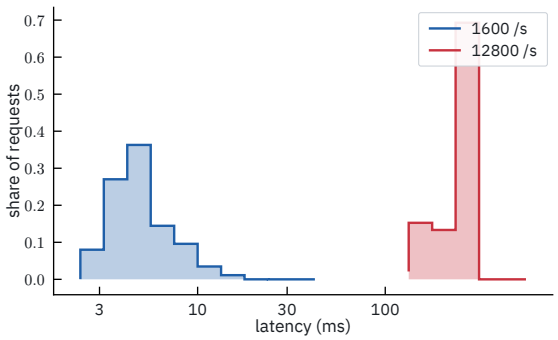


Figure 8. Response Time Distribution

Gatling distribution · share of requests per log-spaced bin

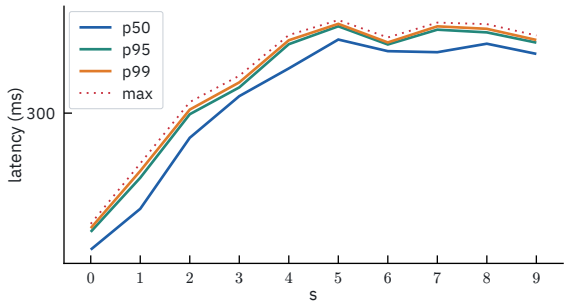


Figure 9. Response Time Percentiles over Time, 12800 req/s

Gatling percentiles-over-time · 1 s bins · successful requests

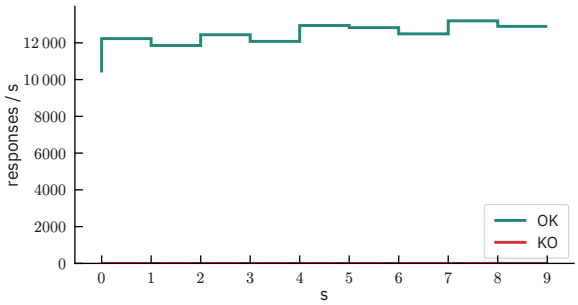


Figure 10. Responses per Second, 12800 req/s

Gatling responses-per-second · OK and KO

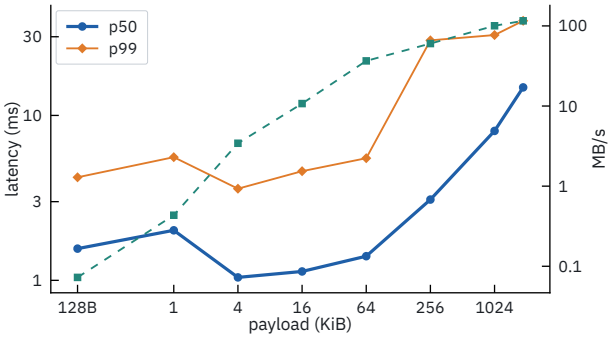


Figure 11. Latency and Bandwidth vs Payload Size

1 client · median of 3 runs · dashed: MB/s (right axis)

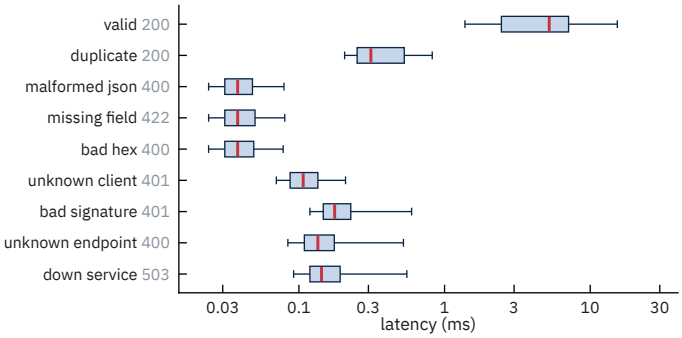


Figure 12. Latency by Request Class

interleaved at random · box p25–p75, whiskers p5–p95 · HTTP status in grey

7 END-TO-END DELIVERY

Format: OpenMessaging Benchmark: fixed publish rate, 1 KiB events, 30 s; publish latency (avg, p50, p99, p99.9, max) and end-to-end latency (p50 … p99.99, max)

Table 5. Publish (HTTP 200) and end-to-end (received by service) latency, ms · median of 3 runs, max over runs

Rate /s	delivered	avg	p50	p99	p99.9	max	p50	p75	p95	p99	p99.9	p99.99	max
			publish latency				end-to-end latency						
500	100.0 %	1.77	0.87	5.99	8.54	25.4	**1.6**	4.26	9.57	**11.0**	14.0	17.3	27.5
1000	100.0 %	5.62	5.38	15.5	43.3	125	**10.1**	12.0	15.4	**28.7**	111	133	165
2000	100.0 %	5.29	5.02	11.8	37.7	180	**9.4**	11.0	13.8	**37.9**	156	176	268
4000	100.0 %	6.95	5.44	42.5	219	2127	**12.2**	30.1	375	**411**	424	427	15893

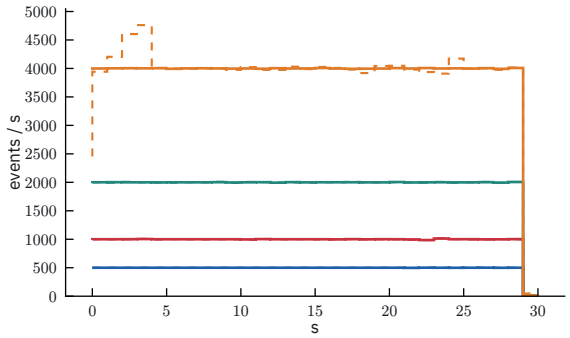


Figure 13. Publish and Consume Rate over Time

solid: publish · dashed: consume · 1 s windows · 500 /s · 1000 /s · 2000 /s · 4000 /s

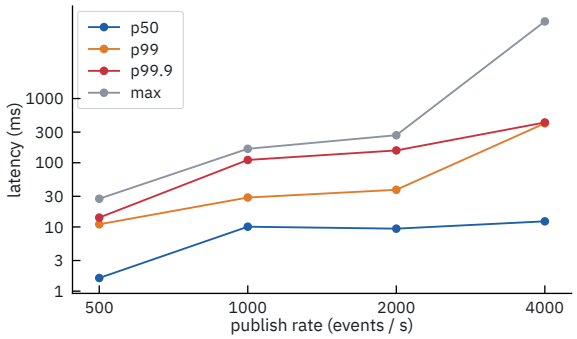


Figure 14. End-to-End Latency vs Publish Rate

percentiles over the whole 30 s run

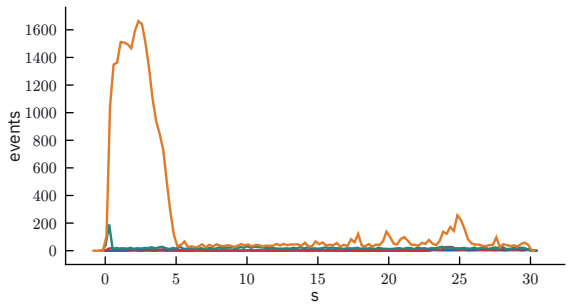


Figure 15. Backlog over Time

PENDING + PROCESSING rows every 250 ms (OMB "backlog") · 500 /s 1000 /s 2000 /s 4000 /s

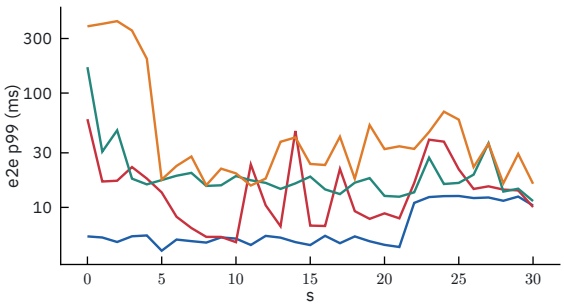


Figure 16. End-to-End p99 per Window

1 s windows · 500 /s 1000 /s 2000 /s 4000 /s

8 DELIVERY AND FAULT METRICS

Format: Hookdeck destination metrics: events failure ratio, average attempts per event, max queue depth, max age of the oldest pending event

Table 6. Delivery metrics by scenario

Scenario	Events	Failure ratio	Avg. attempts	Max depth	Max age (s)
steady 500/s, healthy service	45000	0.00 %	1.00	7	0.03
steady 1000/s, healthy service	90000	0.00 %	1.00	133	0.16
steady 2000/s, healthy service	180000	0.00 %	1.00	470	0.27
steady 4000/s, healthy service	360000	0.00 %	1.00	63397	15.89
5 s outage at 100/s	1400	0.00 %	1.18	747	7.5
attempt failure p = 0.0	300	0.00 %	1.00	—	—
attempt failure p = 0.1	300	0.00 %	1.11	—	—
attempt failure p = 0.25	300	0.33 %	1.36	—	—
attempt failure p = 0.5	300	3.00 %	1.89	—	—
attempt failure p = 0.75	300	21.67 %	2.96	—	—
attempt failure p = 0.9	300	62.00 %	4.16	—	—

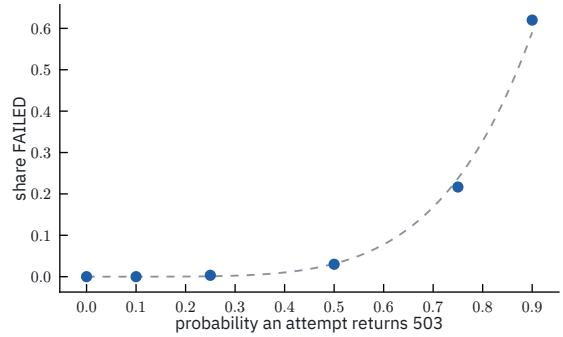


Figure 17. Dead-Letter Share vs Attempt Failure Probability

300 events per point · dashed: theory p^5

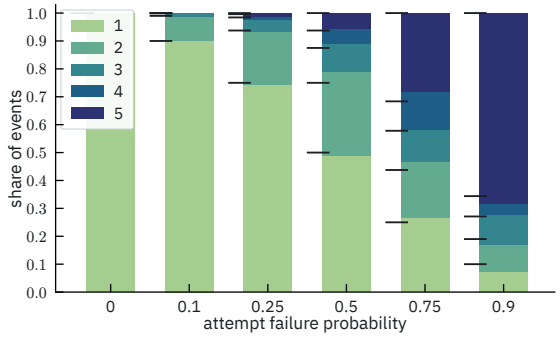


Figure 18. Attempts per Event

cumulative · black ticks: truncated-geometric prediction

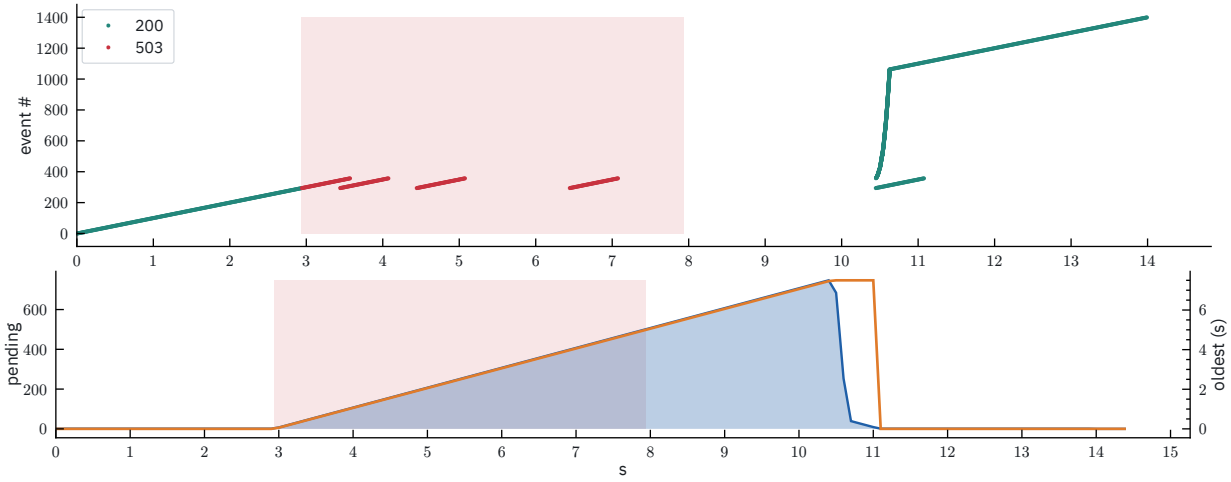


Figure 19. Retry Waves, Queue Depth and Oldest Pending Event During a 5 s Outage

100/s constant arrival · shaded: outage · top: every attempt (event number vs time) · bottom: pending events (blue) and age of the oldest (orange, right axis)

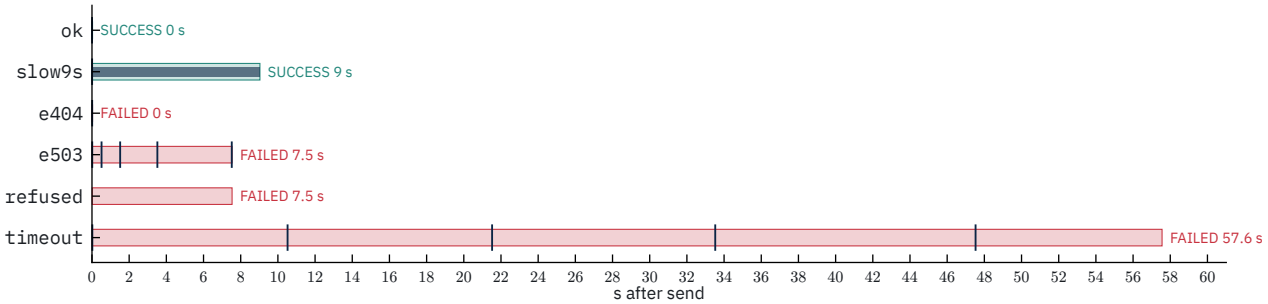


Figure 20. Lifecycle of One Event per Failure Mode

ticks: attempts at the service · dark bars: time inside an attempt · shaded: time to terminal state

9 DISPATCH CAPACITY

Format: Little's law: throughput = concurrency ÷ service time; efficiency = measured ÷ that bound

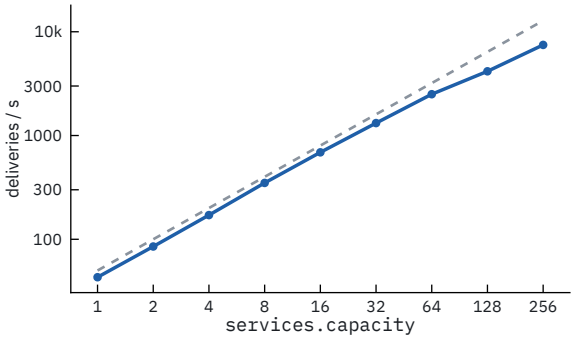


Figure 21. Dispatch Rate vs Service Capacity

backlog parked behind a gate, then released · 20 ms service · dashed: capacity ÷ 20 ms

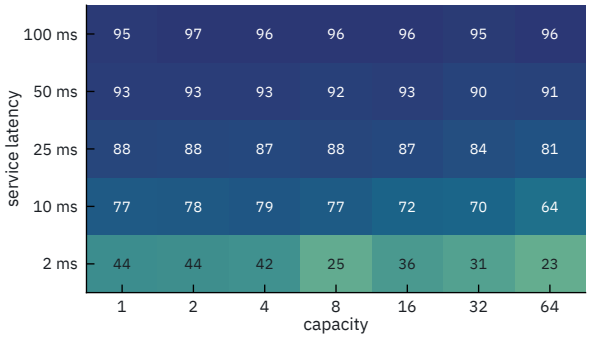


Figure 22. Dispatch Efficiency, Capacity × Service Latency

cell value: measured ÷ (capacity ÷ latency), %

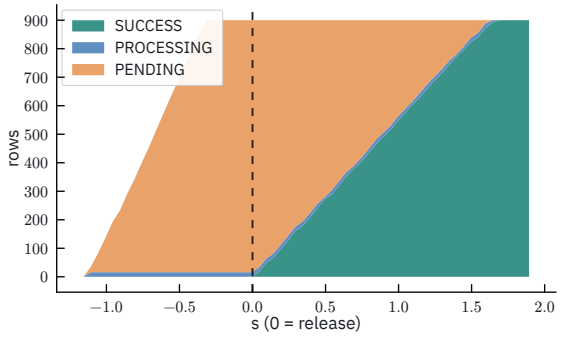


Figure 23. Queue State During Backlog Release

900 events · capacity 16 · 25 ms service · 549/s measured vs 640/s bound

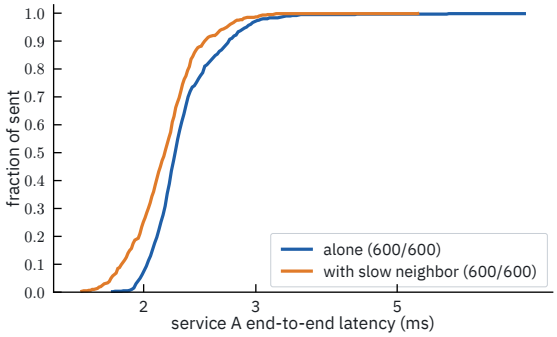


Figure 24. Noisy Neighbour: Fast Service End-to-End Latency

A: 5 ms, capacity 16, 40/s · B: 2 s, capacity 2, 10/s

10 RELIABILITY

Table 7. Stranded events (rows left in PENDING or PROCESSING once the system is idle)

Concurrent clients (4 s burst)	1	2	4	8	16	32
Accepted	3993	7605	15511	23602	29245	33073
Delivered	3993	7605	15511	23602	29245	33073
Stranded	**0**	**0**	**0**	**0**	**0**	**0**

Restart test: 0 stranded before restart, 0 after restart and 5 s idle, 0 after one more event.

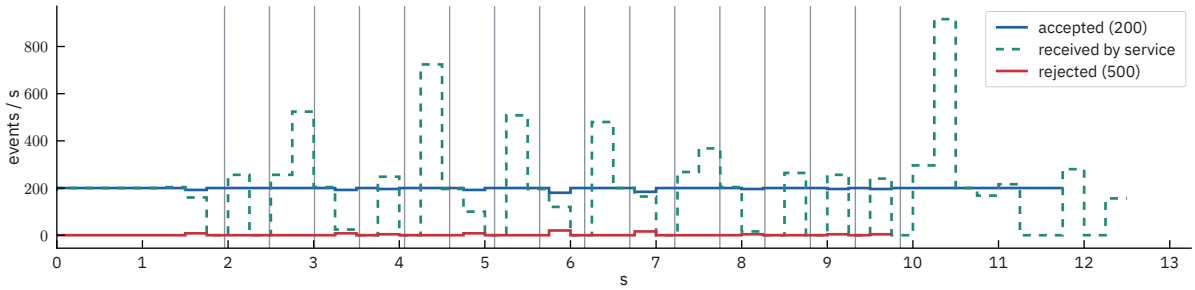


Figure 25. Database Connection Loss Under Load

200/s constant arrival · every wagg connection killed 16× (grey lines) · 2381 accepted, 2400 delivered, 0 lost, 0 stuck

11 RESOURCE UTILISATION

Format: USE method (utilisation, saturation, errors per resource) and RED (rate, errors, duration) — the Google SRE golden signals

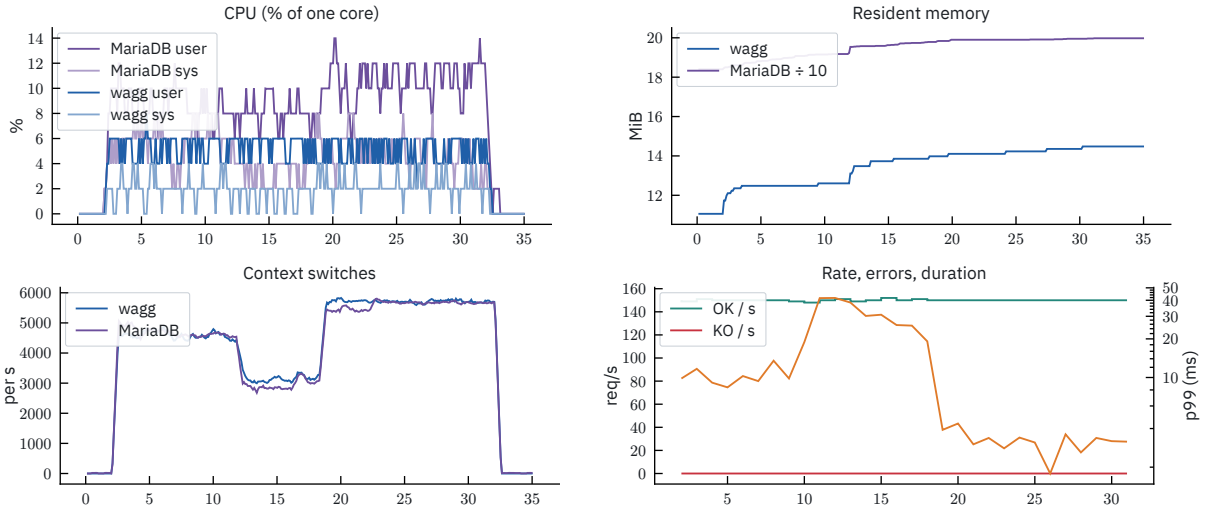


Figure 26. Utilisation and Request Signals over a 30 s Run

150/s constant arrival · `/proc/<pid>/{stat,status,io}` every 100 ms

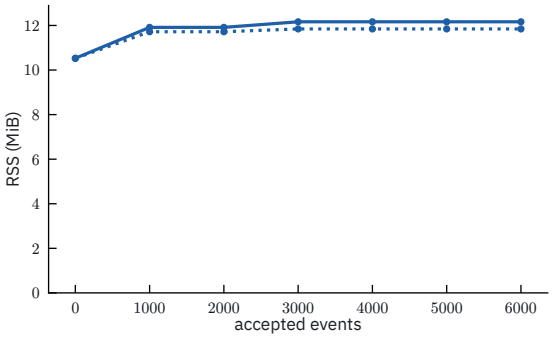


Figure 27. wagg Resident Memory vs Queued Events

service hangs · capacity 1 · solid 16 KiB, dotted 1 KiB payload

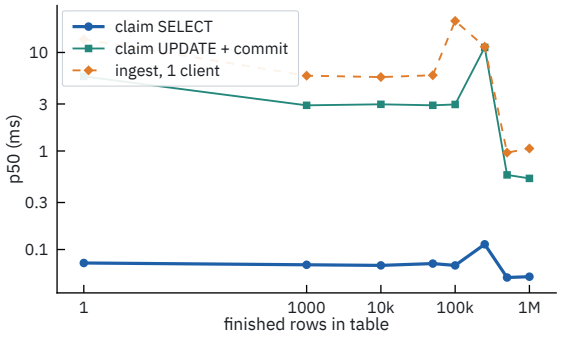


Figure 28. Claim Cost vs Table Size

1 pending row · finished rows pre-filled · claim SELECT (index lookup) and UPDATE + commit

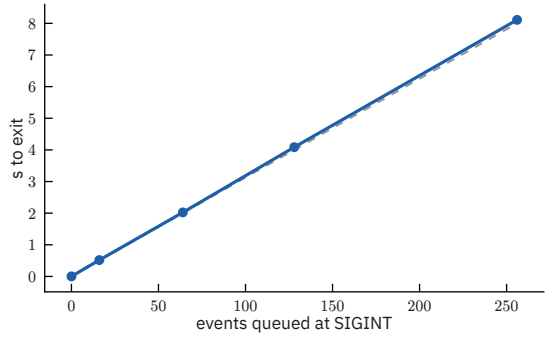


Figure 29. Graceful Shutdown Time vs Queued Events

SIGINT · capacity 16 · 500 ms service · dashed: ⌈n ÷ 16⌉ × 0.5 s

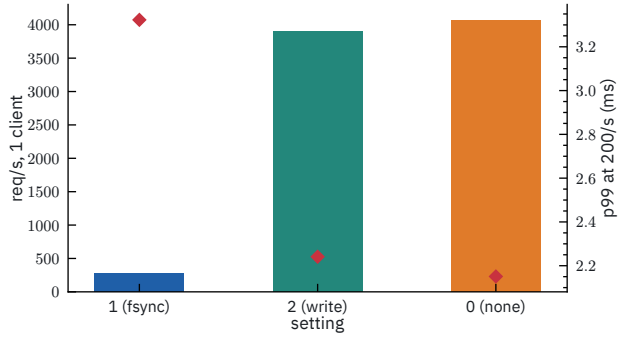


Figure 30. Throughput vs `innodb_flush_log_at_trx_commit`

bars: 1-client req/s · red: p99 at 200/s (right axis)

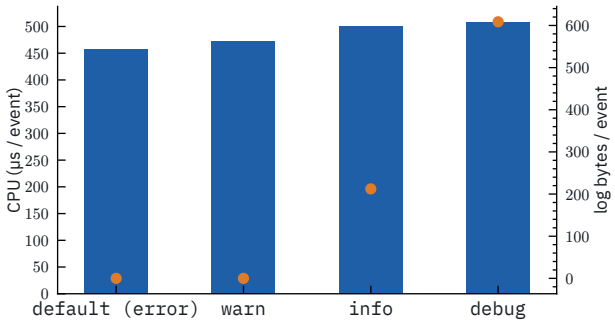


Figure 31. Logging Cost by `RUST_LOG` Level

200/s · bars: wagg CPU per event · dots: log bytes per event (right axis)

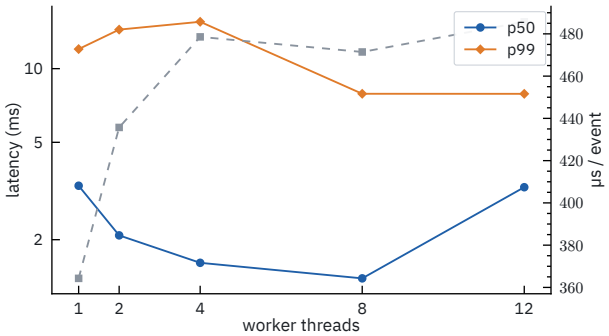


Figure 32. Latency vs Tokio Worker Threads

200/s · `TOKIO_WORKER_THREADS` · dashed: CPU per event (right axis)

12 CPU AND DATABASE PROFILE

Format: on-CPU flame graphs (B. Gregg) from `perf record`; `perf stat` counters; `strace -c` syscall counts

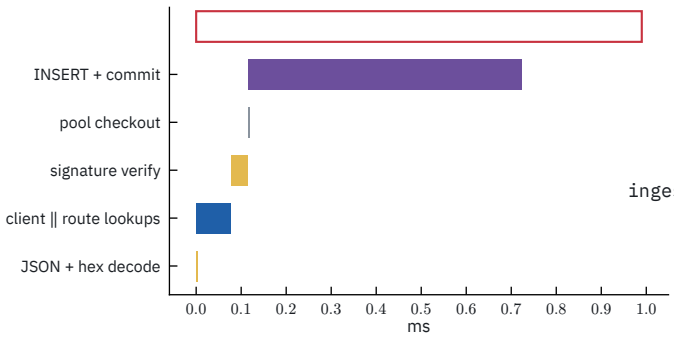


Figure 33. Ingest Critical Path

p50 per step, uncontended · red outline: measured HTTP p50, 1 client

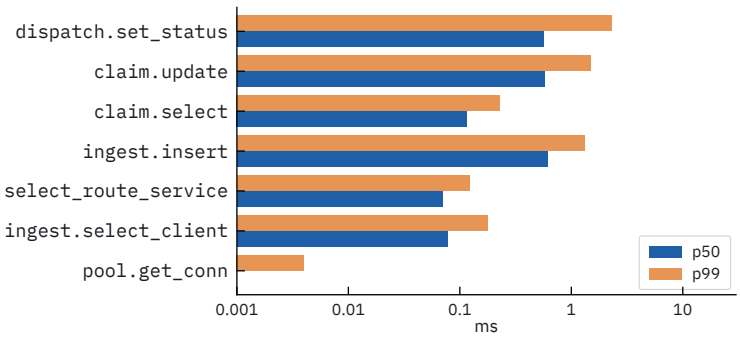


Figure 34. SQL Statement Latency

single connection · wagg stopped · p50 and p99

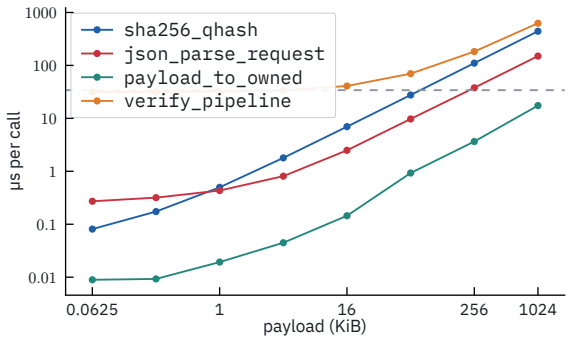


Figure 35. CPU Cost of Per-Request Work vs Payload

micro-benchmarks · median of 60 samples · dashed: Ed25519 verify

Table 8. Per-request CPU and SQL costs (p50)

Operation	µs
ed25519_verify_strict	34.1
ed25519_key_decompress	3.1
ed25519_sign_client	13.2
hex_decode_id_nonce	0.3
JSON + hex decode	0.6
client || route lookups	77.0
signature verify	37.4
pool checkout	1.0
INSERT + commit	607.0

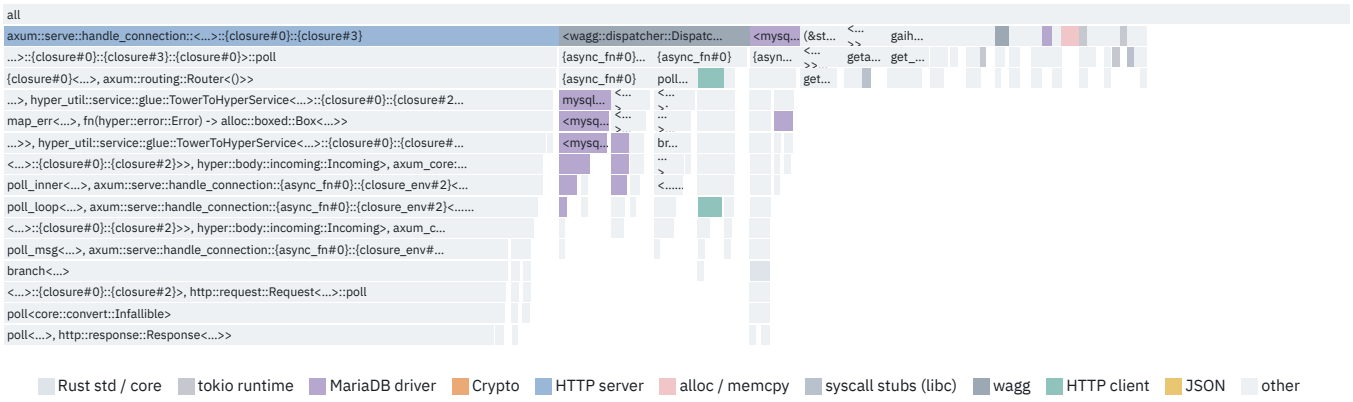


Figure 36. wagg On-CPU Flame Graph

24221 samples · 1999 Hz · 6.4 s · frame-pointer build · 16 clients · runtime frames at the root trimmed · width = share of CPU

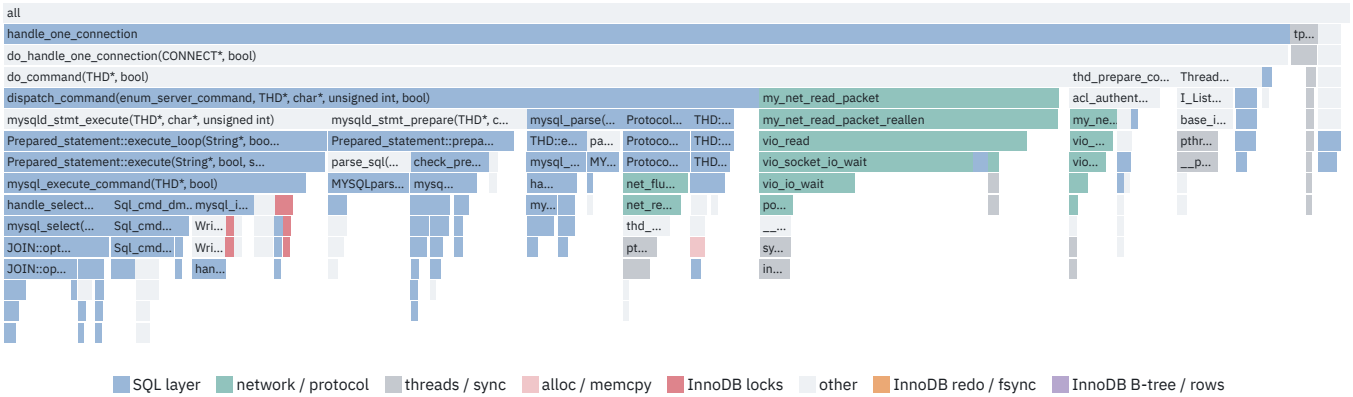


Figure 37. MariaDB On-CPU Flame Graph

14284 samples · 999 Hz · frame-pointer stacks, debuginfod symbols · same load

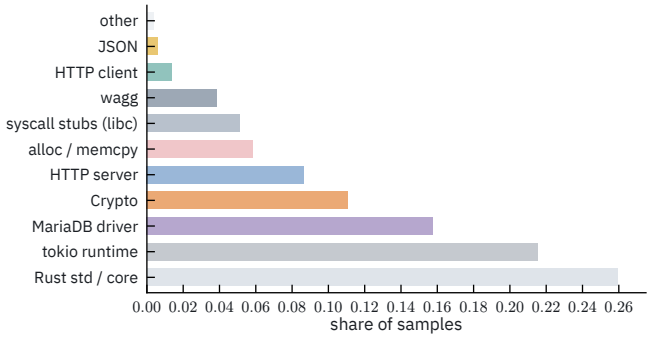


Figure 38. wagg CPU by Component
each sample attributed to its nearest identifiable frame

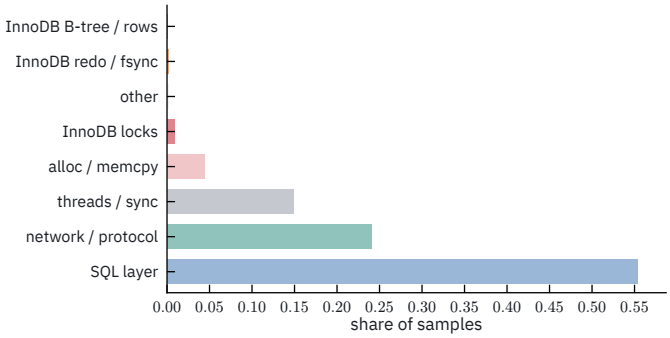


Figure 39. MariaDB CPU by Subsystem
each sample attributed to its nearest identifiable frame



Figure 40. System Calls per Event
`strace -c -f` · counts are exact, times inflated by ptrace

Table 9. wagg counters (`perf stat`, user space), 6.0 s, 24529 events

Counter	Value
IPC	0.7404
cycles / req	1189750
instructions / req	880857
cache misses / req	29577
branch misses / req	8690
branch miss %	10.14
cache miss %	33.76

13 MEMORY PROFILE

Format: heaptrack (KDE) heap profile and allocation flame graphs; page-fault flame graph (B. Gregg, memory flame graphs); `perf stat` memory-hierarchy counters; `/proc/<pid>/smaps_rollup`

Table 10. Heap summary, 1000 events/s for 15 s (heaptrack)

Metric	Total	Per event
Allocation calls	6121954	**408.1**
Temporary allocations	781236	52.1
Bytes allocated (from size histogram)	1214.83 MiB	82.9 KiB
Peak heap	12.52 MiB	—
Peak RSS (with heaptrack)	35.13 MiB	—
Leaked at exit	766.4 KiB	—

Table 11. Top allocation sites (share of all allocation calls)

Allocating function	On behalf of	Share
`deserialize (mod.rs)`	MariaDB driver	19.9 %
`new (error.rs)`	MariaDB driver	11.8 %
`{…} (mod.rs)`	MariaDB driver	4.6 %
`empty (mod.rs)`	MariaDB driver	3.7 %
`{…} (stmt.rs)`	MariaDB driver	3.1 %
`deserialize (de.rs)`	JSON	2.5 %
`build (mod.rs)`	MariaDB driver	2.4 %
`default (sync.rs)`	MariaDB driver	2.4 %

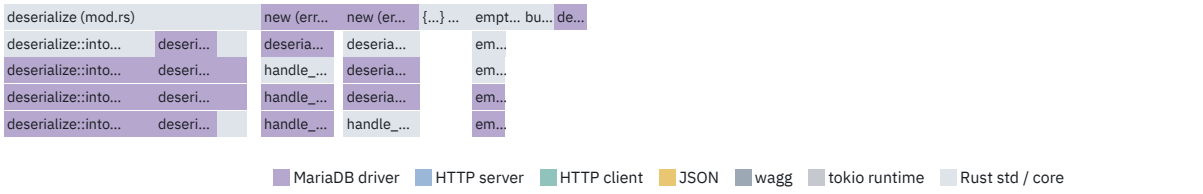


Figure 41. Allocation Flame Graph (bottom-up)
heaptrack caller view · top row: the function that allocates, rows below: its callers · width = share of allocation calls · 1000 events/s

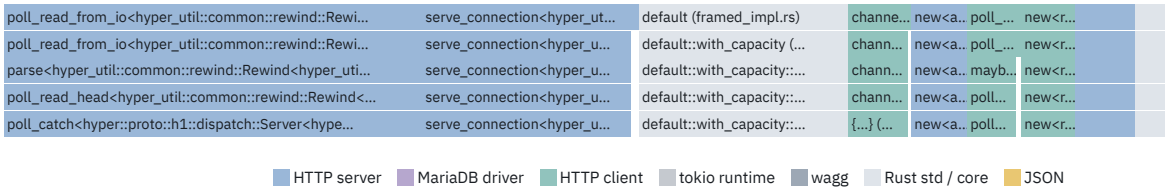


Figure 42. Peak-Memory Flame Graph (bottom-up)
heaptrack caller view · top row: the function that allocates, rows below: its callers · width = share of bytes live at the heap peak · 1000 events/s

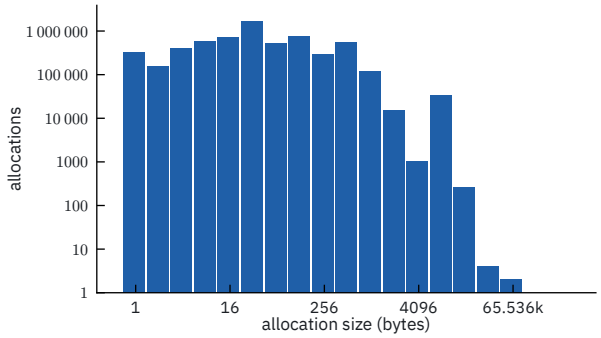


Figure 43. Allocation Size Distribution
heaptrack histogram · power-of-two buckets

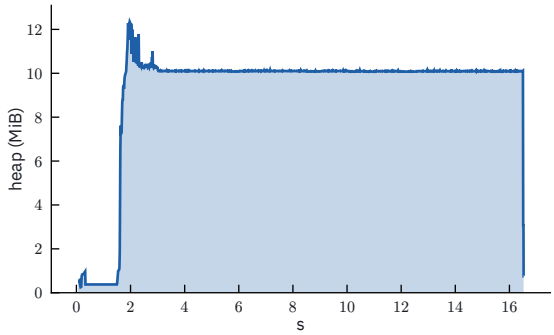


Figure 44. Heap over Time
heaptrack massif output · live heap bytes

Table 12. Memory-hierarchy counters (`perf stat`, user space)

Counter	wagg	MariaDB
instructions / event	860043	1166580
IPC	0.9796	0.689
L1d load miss %	5.48	6.06
dTLB miss %	16.32	14.01
cache miss %	38.25	50.14
L1d misses / event	16517	33192
dTLB misses / event	537	1046

Page-fault sampling (`perf record -e page-faults -c 1`): 109 faults in 4 s at 1000 events/s, i.e. 0.027 per event once warm.

Table 13. Resident memory (`smaps_rollup`, MiB) and buffer pool

Field	wagg	MariaDB
`Rss`	15.8	896.3
`Pss`	12.3	882.3
`Anonymous`	6.0	870.3
`Private_Dirty`	6.0	870.4
`Private_Clean`	6.2	3.8
`Shared_Clean`	3.7	22.2
Memory_used (MiB)	—	2386.6
buffer pool data (MiB)	—	280.5
buffer pool dirty (MiB)	—	29.17
buffer pool pages used (%)	—	13.83
buffer pool hit ratio (%)	—	99.94

14 DISK I/O PROFILE

Format: `iostat -x` (sysstat) device columns from `/proc/diskstats`; InnoDB write counters; fsync latency histogram in the bcc `biolatency` layout; 4 KiB write + `fdatasync` device probe (fio-style sync write)

Table 14. Device statistics on `dm-0` (dm-0 → nvme1n1p3), mean over the steady part of a 12 s run

Events/s	w/s	writes / event	wkB/s	wareq-sz (kB)	w_await (ms)	aqu-sz	%util	r/s
250	3188	12.75	30050	9.4	0.1	0.31	30.5	0
1000	7438	7.44	66954	9.0	0.11	0.85	82.8	0
4000	4387	1.1	39517	9.0	0.18	0.79	74.8	8

Table 15. Write volume per accepted event

Events/s	redo log B	fsyncs	mariadbd write syscalls	mariadbd B to storage	device B	amplification
250	1253	3.01	2.98	122014	123086	**481×**
1000	1199	1.85	1.84	70622	68561	**268×**
4000	1095	0.27	0.27	11105	10116	**40×**

redo log B: `Innodb_os_log_written`; fsyncs: `Innodb_data_fsyncs`; mariadbd B: `/proc/<pid>/io write_bytes`; device B: `dm-0` write bytes ÷ events (includes other writers and file-system metadata); amplification = device B ÷ 256 B payload.

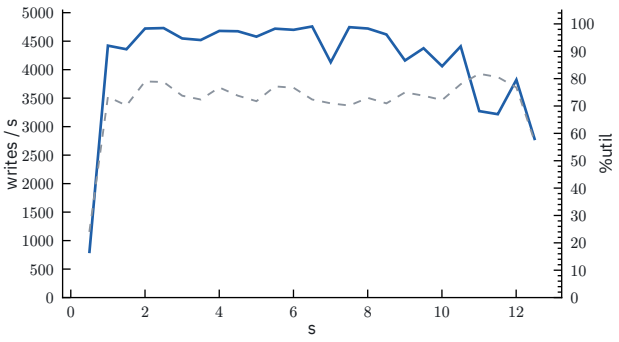


Figure 45. Device Write Activity over Time, 4000 events/s
`dm-0` · 0.5 s samples · solid: writes/s · dashed: %util (right axis)

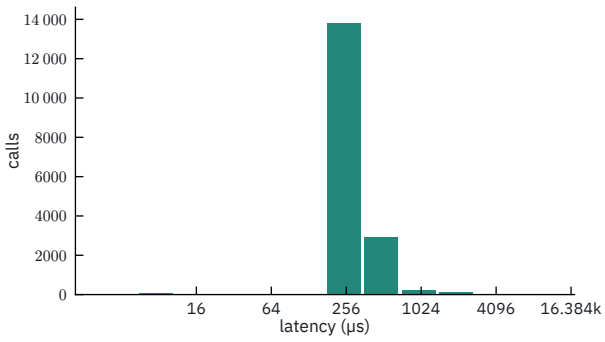


Figure 46. fsync Latency Histogram
`fsync` / `fdatasync` calls of mariadbd at 1000 events/s · log2 buckets of µs

Table 16. fsync latency by file (ms) and device baseline

File	calls	p50	p90	p99	p99.9	max
`ib_logfile0`	17103	0.48	0.54	1.95	4.82	10.0
`queries.ibd`	72	0.01	0.46	0.79	0.92	0.92
`other (28 files)`	36	0.61	1.31	1.31	1.31	1.31
4 KiB write + fdatasync probe	2000	0.44	0.48	1.55	4.94	6.17

15 PARAMETER MEASUREMENT INFORMATION

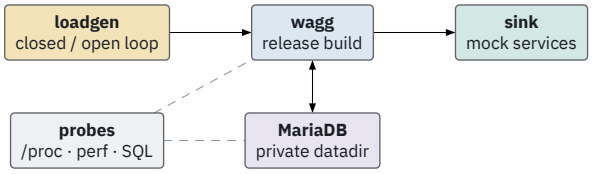


Figure 47. Test setup (private network namespace)

Table 17. Test conditions

Item	Condition
CPU	AMD Ryzen 7 260 w/ Radeon 780M Graphics · 29.2 GiB
Pinning	wagg + MariaDB: CPUs 0-11 load + sink: 12-15
Kernel	7.2.7-zen1-1-zen
Database	mariadbd Ver 13.0.2-MariaDB · `--no-defaults` · 2 GiB buffer pool · `flush_log_at_trx_commit = 1`
Storage	btrfs rw, noatime, compress=zstd:3 · datadir copy-on-write disabled
Toolchain	rustc 1.98.1 · `--release`
Workload	256 B JSON payload, one service, capacity 64, 0 ms service unless stated
Logging	`RUST_LOG` unset (errors only) unless stated
Repetition	3 runs per point; TYP = median

Table 18. Measurement methods and the reporting format they follow

Quantity	Method	Reference format
Capacity	closed loop: N virtual users sending back to back	k6 VUs, USL
Latency under load	open loop on a fixed schedule; latency from the intended send time; HdrHistogram, 3 significant digits	wrk2, HdrHistogram plotter
Latency views	response-time ranges, distribution, percentiles and responses per second over time	Gatling HTML report
Pass / fail	criteria on percentiles and failure rate	k6 thresholds
End-to-end	publish rate, consume rate, backlog, publish and end-to-end percentiles	OpenMessaging Benchmark
Delivery health	failure ratio, attempts per event, queue depth, oldest pending	Hookdeck metrics
Saturation	gated backlogs against capacity ÷ service time	Little's law
Resources	`/proc/<pid>/{stat,status,io}` every 100 ms	USE method, RED, golden signals
CPU	`perf record` (DWARF / frame pointers), `perf stat`, `strace -c`	flame graphs
Heap	every allocation recorded with its backtrace (`LD_PRELOAD`)	heaptrack
Page faults, caches	`perf record -e page-faults`, `perf stat` L1d / dTLB / cache counters	memory flame graphs
Disk	`/proc/diskstats` deltas; InnoDB counters; `strace --seccomp-bpf -T` on fsync	iostat -x, biolatency, fio
Correctness	final state of every accepted event; connection kills; restarts	fault injection

Reproduce: `./test/run.sh` builds both crates in release mode, runs every experiment and renders this document. Raw results are in `test/data/`.

16 DETAILED DESCRIPTION



Figure 48. Event states (`SUCCESS` and `FAILED` are terminal)

Table 19. `queries` indexes

Index	Used by
`PRIMARY (seq)`	status writes; append-only inserts
`dedup (client_id, event_id, service_id)`	duplicate detection at ingest
`work (service_id, status, seq)`	claim: oldest `PENDING` rows of one service

1. The listener commits the `INSERT`, then rings the service's doorbell. A doorbell stores at most one wake-up, so repeated rings collapse.
2. Each service has exactly one claimer. It waits for a free slot (≤ `capacity`), then takes every free slot.
3. It reads the oldest `PENDING` rows of its service through the `work` index and marks them `PROCESSING` in one `UPDATE`. No other session changes those rows, so no locks are needed.
4. Each row is delivered by its own task, which keeps its slot until the final status is stored. Status writes retry until they succeed.
5. When a claim returns nothing, the claimer sleeps until the next ring, or at most 5 s. Rows left from a previous run are claimed on start-up by the same loop.
6. If a claim fails, rows it may have marked are found again by reading `PROCESSING` rows that no task owns.

17 REFERENCES

Source	URL
k6 end-of-test summary	`https://grafana.com/docs/k6/latest/results-output/end-of-test/`
k6 thresholds	`https://grafana.com/docs/k6/latest/using-k6/thresholds/`
Gatling HTML reports	`https://docs.gatling.io/reference/stats/reports/oss/`

Source	URL
wrk2 (coordinated omission)	https://github.com/giltene/wrk2
HdrHistogram	https://github.com/HdrHistogram/HdrHistogram_py
OpenMessaging Benchmark	https://openmessaging.cloud/docs/benchmarks/
Universal Scalability Law	https://cran.r-project.org/web/packages/usl/vignettes/usl.pdf
Hookdeck metrics	https://hookdeck.com/docs/metrics
Tail latencies in DB benchmarks	https://arxiv.org/abs/2107.11607
heaptrack	https://github.com/KDE/heaptrack
Memory flame graphs	https://www.brendangregg.com/FlameGraphs/memoryflamegraphs.html
iostat (sysstat)	https://man7.org/linux/man-pages/man1/iostat.1.html
biolatency (bcc)	https://github.com/iovisor/bcc/blob/master/tools/biolatency_example.txt